ALU project — what we need to build

Style: copy how the existing files look. Every datapath = structural (instantiate modules, no `+` or `&` operators). Every control = FSM like `cu_booth.sv`.

Fixed interface (from the requirements)

```systemverilog
module alu_top (
    input  logic        clk,
    input  logic        rst_n,
    input  logic [3:0]  opcode,
    input  logic        start,
    input  logic  [7:0] A,
    input  logic  [7:0] B,
    output logic  [7:0] result,
    output logic        done,
    output logic        Z,   // zero flag     — result == 0
    output logic        N,   // negative flag — result[7]
    output logic        V    // overflow flag — signed arithmetic overflow
);
```

> Z, N, V are only meaningful for ADD, SUB, MUL, DIV. For logic/shift ops just let them be 0.

Opcodes

opcode	operation
`0000`	ADD
`0001`	SUB
`0010`	MUL
`0011`	DIV
`0100`	AND
`0101`	OR
`0110`	XOR
`0111`	LEFT SHIFT (A << B)

opcode	operation
`1000`	RIGHT SHIFT (A >> B)

Who builds what

> fill in names

File	What it does	Builds on	Person
`add_sub.sv`	ADD and SUB	`adder.sv`, `gates.sv`	
`logic_unit.sv`	AND, OR, XOR	`gates.sv`, `mux.sv`	
`shift_unit.sv`	LEFT SHIFT, RIGHT SHIFT	`register.sv`, `counter_n_bits.sv`	
`csa.sv`	carry-save adder (for DIV)	`gates.sv`	
`lut_quotient.sv`	quotient digit selector (for DIV)	`gates.sv`	
`cu_srt4.sv`	division FSM	—	
`srt4_div.sv`	division datapath	`csa.sv`, `lut_quotient.sv`, `register.sv`, `adder.sv`, `counter_n_bits.sv`	
`flags.sv`	Z, N, V flag logic	`gates.sv`	
`cu_alu.sv`	master FSM	—	
`alu_top.sv`	top level, wires everything	everything above + `booth.sv`, `mux.sv`	

The files

`add_sub.sv` — Addition / Subtraction

Wrap `adder.sv`. When `sub=1`, XOR operand B and set `cin=1` — same trick already used inside `booth.sv` for negation. Output goes straight to `alu_top`.

`logic_unit.sv` — AND / OR / XOR

Three parallel gate arrays (one per operation), then a mux tree selects the right one based on a 2-bit op signal. No clock needed.

`shift_unit.sv` — Left shift and right shift

Use `register.sv` in shift mode (it already supports both directions via `shift_dir`). Drive it with a counter for B cycles. `lshift.sv` is already in the codebase and can be referenced. Assert `done` when the counter reaches B.

`csa.sv` — Carry-Save Adder

Takes 3 inputs, gives back sum + carry without resolving the carry chain. Used every iteration inside the SRT-4 divider. Build it as a `generate` loop of full-adder cells from `gates.sv`.

`lut_quotient.sv` — Quotient digit selector

Given the top bits of the partial remainder and divisor, outputs a digit from {-2, -1, 0, +1, +2}. Pure `always_comb` case block — a ROM in disguise, behavioural here is fine.

`cu_srt4.sv` — Division FSM

Exact same pattern as `cu_booth.sv`. Outputs control word `c[9:0]`. States:

```
IDLE → LOAD → NORMALIZE → ITERATE → CHECK → FINALIZE → CORRECT → OUTPUT → STOP
```

`srt4_div.sv` — Division datapath

Mirror of `booth.sv` but for SRT-4. Instantiates `csa`, `lut_quotient`, registers, `adder`, `counter`. Controlled entirely by `cu_srt4`. A and B arrive as direct inputs (no serial inbus loading needed here).

`flags.sv` — Status flags

Combinational. Takes the final result and computes:

- `Z = ~(result[0] | result[1] | ... | result[7])` — NOR of all bits, use `gates.sv`
- `N = result[7]`
- `V` = overflow: input signs were equal but result sign differs — a few gates

`cu_alu.sv` — Master FSM

Decodes opcode, asserts `start` on the right sub-unit, waits for its `done`, then registers the result and flags. States:

```
IDLE → DISPATCH → WAIT → LATCH → DONE
```

`alu_top.sv` — Top level

Instantiates every module above plus `booth.sv` (MUL). A and B are wired directly to all units simultaneously. A mux tree (built from `mux.sv` instances) selects which unit's output goes to `result` based on opcode. `flags.sv` sits at the end and always reads from `result`.

What already exists and can be reused as-is

File	Used for
`adder.sv`	ADD, SUB (via `add_sub.sv`), final step of DIV
`booth.sv` + `cu_booth.sv`	MUL — **no changes needed**
`register.sv`	shift unit, DIV datapath
`counter_n_bits.sv`	shift unit loop, DIV iteration counter
`lshift.sv`	can be used inside `shift_unit.sv` for the left shift case
`gates.sv`, `mux.sv`, `dff.sv`, `jkff.sv`, `buffer.sv`	everywhere

Build order

```
1. csa, lut_quotient, add_sub, logic_unit, shift_unit, flags   ← all parallel
2. cu_srt4
3. srt4_div
4. cu_alu
5. alu_top
```

`srt4_div` is the only file that blocks on other new files. Everything in step 1 is independent.